

作業系統期中考複習筆記

Ch1–Ch5 完整整理 | 2011–2016 歷屆考題分析

⚡ 超高頻必考主題（每年都考！）

1. 中斷機制 I/O 七步驟流程圖
2. **API 呼叫 System Call**（必提四關鍵字）
3. 行程狀態（各情境對應狀態）
4. **fork()** 程式輸出分析
5. **Shared memory** 程式填寫（shmget/shmat/shmdt/shmctl）

Chapter 1 — Introduction

1.1 作業系統的角色

- OS 是使用者與硬體之間的**中介者（intermediary）**
- 電腦系統四大組成：硬體、OS、應用程式、使用者
- OS 目標：使用者角度（方便）、系統角度（效率）

1.2 ★★★★★ 中斷機制（每年必考）

關鍵名詞

名詞	說明
Interrupt-request line	CPU 在每條指令後都會檢查此線
Interrupt vector（中斷向量表）	儲存各中斷服務程式的位址
Trap（軟體中斷）	由軟體錯誤或 system call 觸發
Interrupt handler	處理特定中斷的程式碼

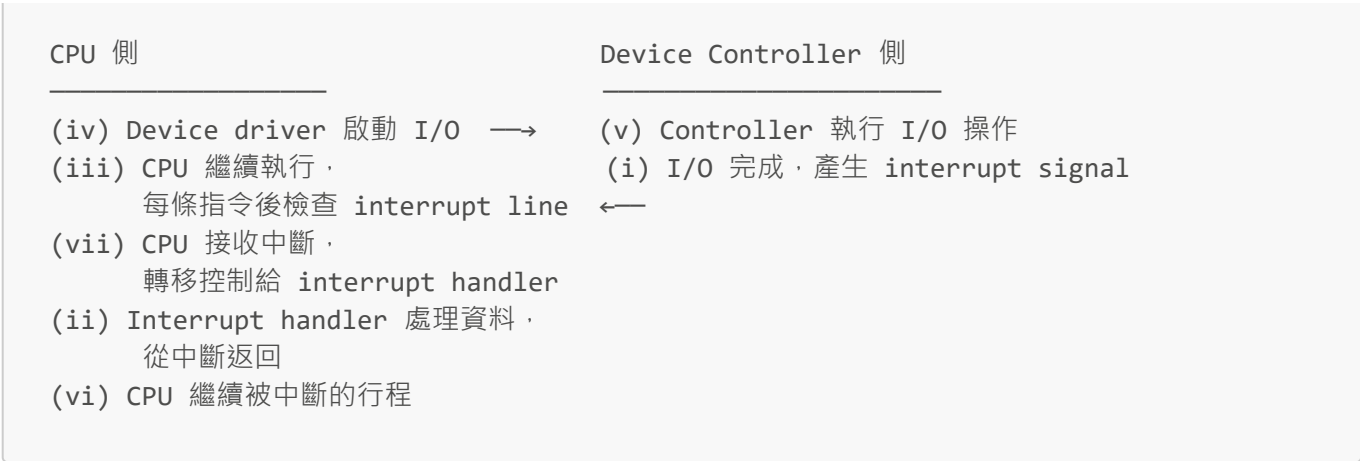
Interrupt Vector 的好處（2011, 2013, 2014 考）

直接查表找到對應的 interrupt-specific handler，不需逐一搜尋，**加速中斷處理速度**。

Interrupt-based vs Busy-waiting

- **Busy-waiting（輪詢）**：CPU 持續循環檢查，浪費 CPU 時間，CPU 大量閒置
- **Interrupt-based**：CPU 繼續執行其他工作，等設備完成才發中斷，**CPU 使用率高**

★ 中斷驅動 I/O 七步驟（每年必考的流程圖題）



記法：CPU 側順序 iv → iii → vii → ii → vi；Device 側 iv → v → i

1.3 Device Controller vs Device Driver

比較	Device Controller	Device Driver
類型	硬體元件	軟體（OS 的一部分）
功能	讀寫儲存於設備中的資料，有 local buffer	告訴 OS 如何與設備溝通的方法

OS 如何支援新設備？ → 安裝新的 **device driver**（驅動程式）

1.4 儲存體系層次（由快到慢）



1.5 多處理器系統

	非對稱（Asymmetric）	對稱（SMP）
結構	Master + Slaves	所有 CPU 地位相同
Master	控制系統，指派工作	無 master

	非對稱 (Asymmetric)	對稱 (SMP)
共享	Main memory	Main memory
現代使用	少	主流

1.6 ★ Multiprogramming vs Time-sharing

	Multiprogramming	Time-sharing
目的	提高 CPU 使用率	提供互動性
關係	基礎	Multiprogramming 的延伸
切換時機	等 I/O 時	頻繁 (time quantum)
互動性	不需要	需要 · response time 短

1.7 ★ 雙模式操作 (Dual-Mode)

- **mode bit = 0** → Kernel mode (可執行所有指令)
- **mode bit = 1** → User mode (不可執行特權指令)

如何保護 OS：Privileged instructions 只能在 kernel mode 執行。User mode 嘗試執行 → hardware trap → OS 處理，防止惡意/錯誤程式破壞系統。

模式切換時機：interrupt / trap / system call → kernel mode；OS 完成 → user mode

Chapter 2 — System Structures

2.1 OS 提供的服務

為使用者：使用者介面 (CLI/GUI)、程式執行、I/O、檔案系統、通訊、錯誤偵測

為系統效率：資源分配、記帳 (accounting)、保護與安全

2.2 CLI 兩種實作方式 (2015 Q3 考)

方式一：命令內建在直譯器中

- ☒ 優點：速度快
- ☒ 缺點：直譯器體積大；少用指令浪費記憶體；需修改直譯器才能新增指令

方式二：透過系統程式 (UNIX 採用)

- 直譯器不含指令碼，搜尋 PATH 找到系統程式，載入執行
- ☒ 優點：直譯器體積小；可輕易新增指令
- ☒ 缺點：需搜尋和載入，速度較慢

使用高階語言（**High-level Language**）實作 OS 的優點（2015 Q2）：

1. 程式碼容易閱讀和理解
2. 可以更快速撰寫
3. 可與他人分享、用 compiler/debugger 除錯
4. **OS 容易移植（port）** 到其他硬體平台 ← 最重要

2.3 ★★★★★ System Call 透過 API 呼叫（每年必考！）

考試必提的四個關鍵字

- ① Software interrupt（軟體中斷）
 - ② Dual-mode（雙模式）
 - ③ System call number / index（系統呼叫號碼）
 - ④ Table of code pointer（程式碼指標表）

完整流程

1. 使用者程式呼叫 API 函數（如 `read()`）
2. API 函數觸發 **software interrupt（trap）**，CPU 切換到 **kernel mode（dual-mode）**
3. 將 **system call number（index）** 放入 EAX 暫存器
4. CPU 查詢 **table of code pointer**，找到 kernel 函數位址
5. 執行對應的 kernel 服務
6. 完成後 CPU 切回 **user mode**，回傳結果

為何偏好 API 而非直接呼叫 system call？（每年必考）

1. **Portability（可攜性）**：API 在不同 OS 提供一致介面
2. **Simplicity（簡便性）**：不需了解底層 system call 細節

2.4 System Call 參數傳遞三種方式（2013, 2014 考）

1. **Registers（暫存器）**：最簡單，但數量受限
2. **Memory block** ← 適合多參數，Linux/Solaris 使用：參數存入記憶體，位址放入 register
3. **Stack（堆疊）**：程式 push 參數，OS 從 stack pop 取出

2.5 ★ OS 結構比較

架構	優點	缺點
Layered（分層）	易於除錯（逐層驗證）	難定義各層；效能差（多層開銷）
Microkernel（微核心）	可靠、可移植、安全	效能開銷大（user/kernel 間頻繁 message passing）

架構	優點	缺點
Modules (模組化) ★現代主流	彈性高、效率高；模組間可直接溝通	—

Modular 優於 **Layered**：模組間可直接溝通，不需穿越多層
Modular 優於 **Microkernel**：模組在 kernel space，不需 message passing 開銷

2.6 系統開機 (Boot)

- 1. **Bootstrap program** (存在 ROM/EPROM) → 初始化硬體
- 2. Bootstrap → 載入 OS kernel
- 3. PC 兩步驟：簡單 bootstrap → boot block (磁碟第一個區塊) → 完整 kernel

Chapter 3 — Processes

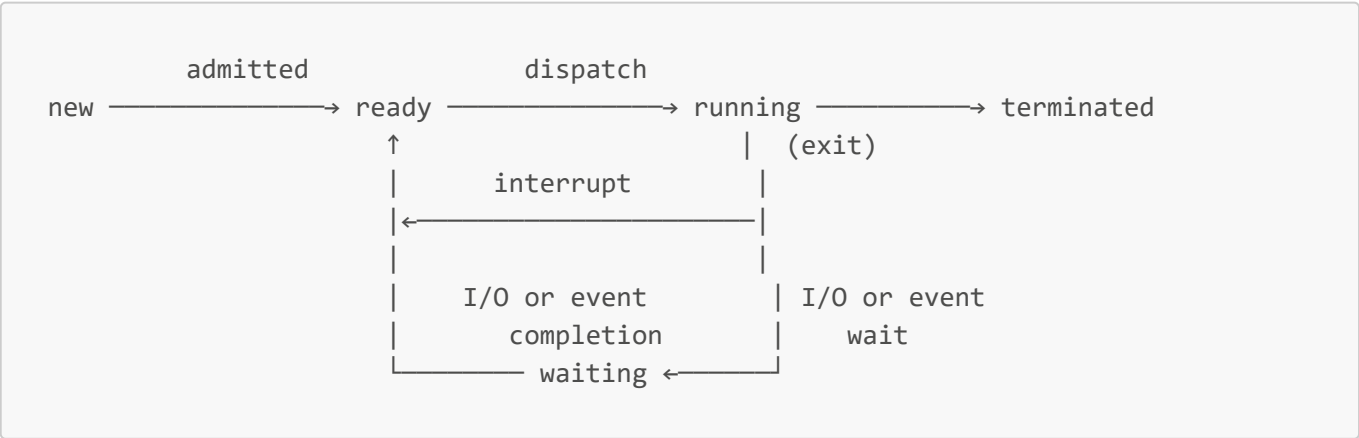
3.1 行程概念

- **程式 (Program)**：磁碟上的被動實體 (passive entity)
- **行程 (Process)**：執行中的程式，主動實體 (active entity)

行程記憶體四個區段

區段	內容
Text	可執程式碼
Data	全域變數 (global variables)
Heap	動態配置記憶體 (malloc/new)，往上成長
Stack	局部變數、函數參數、返回位址，往下成長

3.2 ★★★★★ 行程狀態 (每年必考)



各情境對應狀態 (背起來！)

情境	狀態
被 long-term scheduler 選中 (admitted) 後	ready
在 short-term scheduler 選中之前	ready
被 short-term scheduler 選中 (dispatch) 後	running
呼叫 blocking system call 之前	running
呼叫 blocking system call 之後	waiting
blocking system call 完成後	ready
被 hardware interrupt 中斷後	ready ⚠ (不是 waiting !)
time slice 到期後	ready ⚠ (不是 waiting !)
fork() 建立後的子行程	ready
exit() 呼叫後	terminated

⚠ 易錯：hardware interrupt 和 time slice 到期後都是 **ready**，不是 waiting！

3.3 排程器 (Schedulers)

	Long-term (Job) Scheduler	Short-term (CPU) Scheduler
選什麼	從 job pool 選 jobs 放入記憶體	從 ready queue 選行程給 CPU
頻率	低 (分鐘級)	高 (每~100ms)
速度	可以慢	必須快
UNIX/Windows	沒有 (所有行程直接入記憶體)	有

3.4 ★★ Fork / Exec / Wait / Exit

```

pid_t pid = fork();
// 子行程回傳 0；父行程回傳子 PID(>0)；失敗回傳 -1

if (pid == 0) {           // 子行程
    execlp("ls", "ls", NULL); // 用新程式取代記憶體
} else {                  // 父行程
    wait(NULL);           // 等子行程完成
    printf("done\n");
}

```

重要觀念： fork() 後父子行程有各自獨立的位址空間副本，修改一方完全不影響另一方（包括全域變數）。

移除 wait(NULL) 的後果： 父子並行，輸出不定；子行程結束後成為 **zombie process** (殭屍行程)。

3.5 ★★ 2016 考題 fork() 輸出分析

```
int value = 5; // 全域變數

int main() {
    pid_t pid = fork();
    if (pid > 0) {           // 父行程
        value += 15;
        wait(NULL);
        printf("value = %d\n", value);
    } else if (pid == 0) {   // 子行程
        value += 5;
        printf("value = %d\n", value);
    }
}
```

輸出：

```
value = 10    ← 子行程先印 ( 5+5=10 )
value = 20    ← 父行程後印 ( 5+15=20 · wait 後才印 )
```

3.6 ★★ Shared Memory API (System V)

```
// 1. 建立/取得共享記憶體
int segment_id = shmget(IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);

// 2. 附加 ( attach ) 到行程位址空間
int *shared_memory = (int *) shmat(segment_id, NULL, 0);

// 3. 使用
*shared_memory = 0; // 初始化

// 4. 分離 ( detach )
shmdt(shared_memory);

// 5. 移除
shmctl(segment_id, IPC_RMID, NULL);
```

計算 $2x+3y-5$ 的標準解答 (2015 Q9)

```
int segment_id = shmget(IPC_PRIVATE, 4, S_IRUSR | S_IWUSR);
int *shared_memory = (int *) shmat(segment_id, NULL, 0);
*shared_memory = 0;
```

```

for (i = 2; i > 0; i--) {    // 建立 2 個子行程
    pid = fork();
    if (pid < 0) { fprintf(stderr, "Fork Failed"); exit(-1); }
    else if (pid == 0) {    // 子行程
        if (i == 2) *shared_memory += 2 * x;    // 計算 2x
        if (i == 1) *shared_memory += 3 * y;    // 計算 3y
        shmdt(shared_memory);
        exit(0);
    } else {                // 父行程
        wait(NULL);        // 等待確保順序
    }
}
*shared_memory -= 5;
printf("the result is %d\n", *shared_memory);
shmdt(shared_memory);
shmctl(segment_id, IPC_RMID, NULL);

```

3.7 ★★ Shell 程式填寫 (2013, 2016 考)

```

while (cin >> inputBuffer) { // 或 !feof(stdin) ^ true
    // Step 1: 讀取命令，若最後字元是 '&' 則 background = true

    pid = fork();
    if (pid == 0) {                // 子行程
        exec(inputBuffer);        // 執行指令
    } else if (pid > 0) {          // 父行程
        if (background == false)
            wait(NULL);           // 前景：等子行程
        // 背景：不等，直接繼續
    }
    exit(0);                      // 子行程結束
}

```

3.8 IPC 比較

	Shared Memory	Message Passing
速度	⚡ 快 (設置後不需 kernel)	較慢 (每次需 system call)
適用	大量資料	少量資料；分散式系統
同步	需自行處理	OS 提供機制

Chapter 4 — Multithreaded Programming

4.1 Thread 概念

Thread 自己擁有：Thread ID、Program Counter、Register set、Stack

同行程 **Threads** 共享：Code、Data、Heap、Open files

4.2 ★ Thread 比 Process 便宜的原因（2015, 2016 考）

建立 Thread 不需複製整個位址空間（text/data/heap），只需建立 Thread ID/PC/registers/stack，共享其餘部分。在 Solaris 中，Thread 建立比 Process 快約 **30 倍**。

4.3 ★★ 執行緒模型比較（每年必考）

Many-to-One（多對一）

- 多個 user threads → 一個 kernel thread
- ☒ Thread 管理在 user space，效率高，不需 system call
- ☒ 一個 thread 做 blocking call → **整個行程阻塞**；無法在多處理器平行執行

One-to-One（一對一）★ Linux / Windows

- 每個 user thread → 一個 kernel thread
- ☒ 一個 thread 阻塞不影響其他；可在多處理器**真正平行執行**
- ☒ 每建立 user thread 就建 kernel thread，**overhead** 大；數量可能受限

Many-to-Many（多對多）

- 多個 user threads ↔ 多個 kernel threads（kernel 數量 ≤ user 數量）
 - ☒ 兼具兩者優點；可建立足夠多 user threads；一個阻塞不影響其他
-

4.4 ★ Scheduler Activations（2011, 2012, 2014 考）

解決的問題：Many-to-Many 模型中，user thread 做 blocking call 時，kernel 阻塞整個行程。

LWP（Lightweight Process）：kernel 與 thread library 之間的虛擬處理器。

運作流程（考試必答）：

1. User thread 準備做 **blocking system call**
 2. Kernel 透過 **upcall**（向上呼叫）通知 thread library
 3. Thread library 的 **upcall handler** 儲存被阻塞 thread 的狀態
 4. Thread library 在 **LWP** 上排程另一個可執行的 user thread
 5. Blocking call 完成後，kernel 再次 **upcall** 通知
 6. Thread library 將原 thread 標記為 ready，擇機重新排程
-

Chapter 5 — CPU Scheduling

5.1 排程標準（Scheduling Criteria）

最大化：CPU utilization (使用率) 、Throughput (吞吐量)

最小化：Turnaround time (完成時間) 、Waiting time (等待時間) 、Response time (回應時間)

5.2 ★ 可搶占 vs 不可搶占 (2014 Q7 考)

情況	類型
① running → waiting (I/O request)	非搶占 (必須選)
② running → ready (interrupt 發生)	可搶占 ★
③ waiting → ready (I/O 完成)	可搶占 ★
④ terminated	非搶占 (必須選)

情況 2, 3 是可搶占排程的關鍵點

5.3 排程演算法比較

演算法	描述	優點	缺點
FCFS	先來先服務 · 非搶占	簡單	Convoy Effect
SJF	選 CPU burst 最短的	平均等待時間最短 (optimal)	難以預知 burst 長度
Priority	優先權最高的先	靈活	Starvation (飢餓)
Round-Robin	輪流每個 time quantum	公平 · 好 response time	quantum 過大→FCFS ; 過小→overhead 大

Starvation 解法：Aging (老化) — 隨著等待時間增加 · 逐漸提高行程的優先權

5.4 ★★ 多層反饋佇列 (Multilevel Feedback Queue · 2014, 2015 考)

Q2 (最高優先) : Round-Robin · time quantum = 4ms ← 新行程進入此層
 ↓ (超時被搶占 → 降層)
 Q1 (中優先) : Round-Robin · time quantum = 6ms
 ↓ (超時被搶占 → 降層)
 Q0 (最低優先) : FCFS

規則：

- 新行程進入 **Q2**
- 在 quantum 內完成 → 離開 CPU
- **超時被搶占** → 降到下一層
- 高優先層行程 **搶占** 低優先層行程

等待時間計算：等待時間 = 完成時間 - 到達時間 - CPU burst 總長

考試常用程式碼速查

Shared Memory (System V) 完整範本

```
#include <sys/shm.h>

int segment_id = shmget(IPC_PRIVATE, sizeof(int), S_IRUSR | S_IWUSR);
int *shared_memory = (int *) shmat(segment_id, NULL, 0);
*shared_memory = 0;

pid_t pid = fork();
if (pid == 0) { // 子行程
    *shared_memory += 計算值;
    shmdt(shared_memory);
    exit(0);
} else { // 父行程
    wait(NULL);
}

printf("result = %d\n", *shared_memory);
shmdt(shared_memory);
shmctl(segment_id, IPC_RMID, NULL);
```

Pthreads 基本用法

```
#include <pthread.h>

void *worker(void *param) {
    // 執行工作
    pthread_exit(0);
}

pthread_t tid;
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_create(&tid, &attr, worker, NULL);
pthread_join(tid, NULL);
```

關鍵英文術語速查

中文

英文

中文	英文
中斷	interrupt
中斷向量	interrupt vector
軟體中斷	software interrupt / trap
裝置控制器	device controller
裝置驅動程式	device driver
特權指令	privileged instruction
系統呼叫	system call
程式碼指標表	table of code pointer
行程控制區塊	PCB (Process Control Block)
上下文切換	context switch
殭屍行程	zombie process
飢餓	starvation
老化	aging
輕量級行程	LWP (Lightweight Process)
向上呼叫	upcall
時間量子	time quantum
多層反饋佇列	multilevel feedback queue
可搶占排程	preemptive scheduling
多程式處理	multiprogramming
分時系統	time-sharing

易錯重點整理

1. **hardware interrupt** 後 → **ready** (不是 waiting ! 中斷只是搶占 CPU, 行程沒在等事件)
 2. **time slice** 到期後 → **ready** (不是 waiting ! 到期只是被搶占)
 3. **fork()** 後父子各有獨立的變數副本, 全域變數也是各自獨立的
 4. **System Call** 必答四關鍵字 : software interrupt / dual-mode / system call number (index) / table of code pointer
 5. **Long-term scheduler** : UNIX/Windows 沒有 ; 直接放入記憶體
 6. **Shared memory** 比 **Message passing** 快的原因 : 設置後不需 **kernel** 介入
 7. **Thread** 比 **Process** 便宜的原因 : 不需複製整個位址空間
-

歷屆考題出題規律 (2011–2016)

主題	2011	2012	2013	2014	2015	2016
中斷機制流程圖	✓	✓	✓	✓	✓	✓
System Call via API	✓	✓	✓	✓	✓	✓
行程狀態 (7小題)	✓	✓	✓	—	✓	✓
執行緒模型比較	✓	✓	✓	✓	✓	✓
Shared Memory 程式	✓	✓	—	✓	✓	—
Shell 程式填寫	—	—	✓	—	—	✓
CPU 排程甘特圖	—	—	—	✓	✓	—
Scheduler Activation	✓	✓	—	✓	—	—
OS 結構優缺點	✓	✓	✓	—	✓	✓
fork() 輸出分析	—	✓	—	—	—	✓